TP4 — Convolution directe par blocs

TP

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 1 h 35**

Problématique

Filtrer, c'est convoluer : $y = h * x$. Mais `update()` ne reçoit que 128 échantillons à la fois. Comment calculer exactement la convolution d'un flux continu, bloc par bloc, et combien de coefficients peut-on se permettre en temps réel ?

Objectifs:

- Implémenter un filtre RIF par **convolution directe** dans un objet `AudioStream` ;
- Comprendre la **mémoire entre blocs** : les $M - 1$ échantillons passés indispensables ;
- La ranger dans un **buffer circulaire** (index d'écriture, masque binaire) ;
- Lire à l'oscilloscope des réponses impulsionnelles : retard, écho, moyenne, passe-bas, salle ;
- Mesurer le **coût** (M multiplications par échantillon) et la limite du temps réel.

Prérequis:

- Jour 1 validé (objet `AudioStream` personnalisé, saturation `int16`) ;
- Convolution discrète, réponse impulsionnelle, filtres RIF (TNS hors-ligne).

® **Remarque** : Projet Arduino

`jour2/TP4a_convolution_directe`, en version `etudiant` et `corrige`. On ne modifie que le `.ino` (réglages) et le fichier de l'objet de convolution (`conv_directe.cpp`).

® **Remarque** : Montage et réglages

Oscilloscope : voie 1 sur `LINE OUT` gauche (signal traité), voie 2 sur `LINE OUT` droite (source non traitée), déclenchement sur la voie 2. Tous les réglages se font dans `setup()`, dans le fichier `.ino`, puis on téléverse à nouveau (une vingtaine de secondes) :

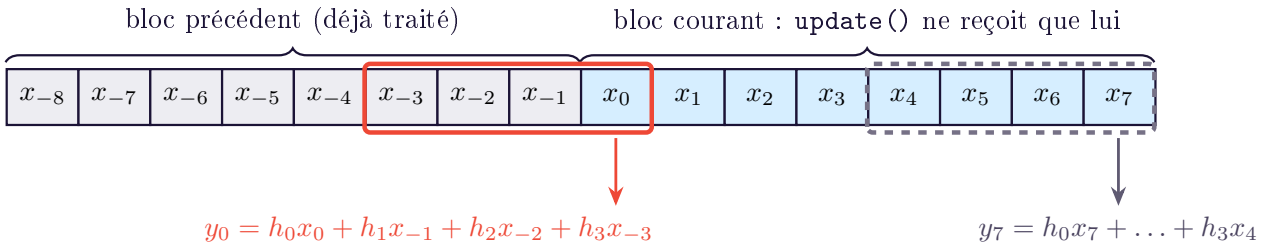
- **source** : gain 1 sur une seule voie du mélangeur `source` : *impulsions* (une toutes les 100 ms : la voie 1 montre directement $h[n]$), *sinus* de test, ou *musique* du PC par USB (*Outils* → *USB Type* → *« Serial + MIDI + Audio »*) ;
- **réponse impulsionnelle** : garder une seule ligne `int M = noyau...(h, ...);` (fonctions de `noyaux.h`) ;
- **charge processeur** : affichée chaque seconde dans le moniteur série, en pourcentage de la durée d'un bloc (2,9 ms).

1 Cours éclair : convolution par blocs

La mémoire entre blocs. Un filtre RIF de M coefficients calcule

$$y[n] = \sum_{k=0}^{M-1} h[k]\, x[n-k].$$

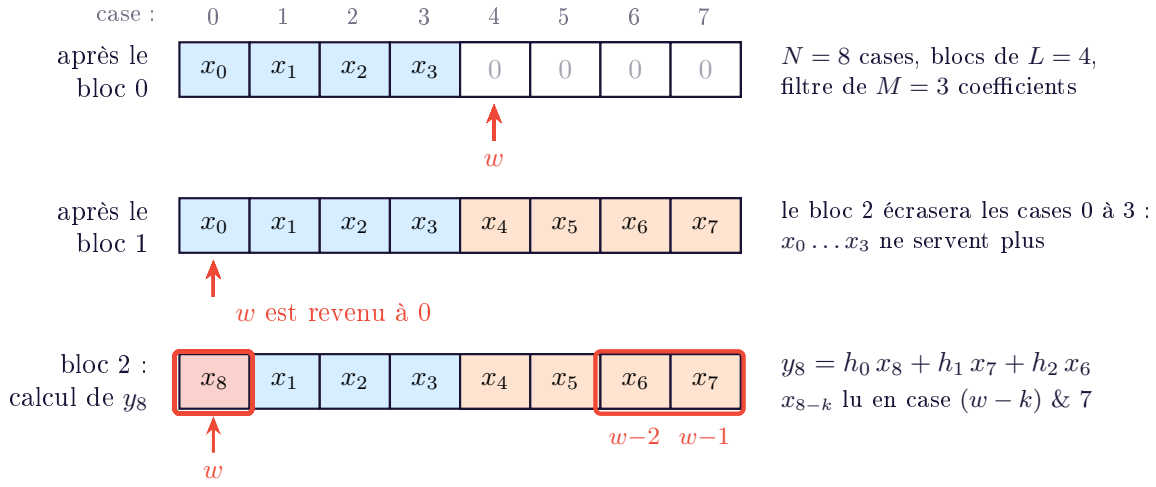
Chaque sortie a besoin de l'échantillon courant *et des* $M-1$ *précédents*. Or `update()` ne voit qu'un bloc de $L = 128$ échantillons : pour les premières sorties du bloc, une partie des entrées nécessaires appartient au bloc *précédent*, déjà traité et rendu à la bibliothèque. Le schéma ci-dessous l'illustre avec des nombres plus petits ($L = 8$, $M = 4$).



Pour y_0, 3 des 4 entrées sont dans le bloc précédent.
En général : il faut **garder les** $M-1$ **derniers échantillons** d'un bloc à l'autre.

La convolution par blocs a donc une **mémoire** : l'objet doit garder, d'un appel de `update()` au suivant, au moins les $M-1$ derniers échantillons reçus.

Le buffer circulaire. On range tous les échantillons reçus dans un tableau de taille fixe `CIRC_N`, parcouru en boucle. Un index d'écriture w indique la case du prochain échantillon ; il avance d'une case par échantillon et revient à 0 après la dernière case. Le nouvel échantillon écrase le plus ancien, qui ne sert plus : rien n'est jamais déplacé. L'échantillon $x[n-k]$ se trouve alors k cases avant w, à l'indice $(w-k)$ modulo `CIRC_N`. Le schéma suit trois blocs successifs dans un buffer de 8 cases ($L=4$, $M=3$).



Avec `CIRC_N` $= 4096 = 2^{12}$, le modulo s'écrit avec un masque binaire : $(w-k)$ & (`CIRC_N` -1) garde les 12 bits de poids faible, et le résultat reste juste quand $w-k$ est négatif (complément à deux).

Coût et budget. M multiplications-accumulations (MAC) par échantillon. À 600 MHz et 44,1 kHz, le budget est de $600 \cdot 10^6 / 44\,118 \approx 13\,600$ cycles par échantillon, *pour tout le traitement*.

2 Convolution avec un buffer circulaire (projet TP4a)

Activité 1

Q1. Pour $M = 256$, combien d'échantillons du bloc précédent faut-il pour calculer $y[0]$? Et $y[127]$? Combien faut-il en garder d'un bloc à l'autre ?

Q2. Pourquoi le buffer doit-il avoir au moins M cases ? Pour $w = 2$ et $k = 5$, que vaut l'indice $(w-k)$ & 4095 ? Quelle case du tableau désigne-t-il ? Pourquoi une taille de 4000 cases ne conviendrait-elle pas avec ce masque ?

Q3. Dans `conv_directe.cpp`, compléter les trois TODO (Q3) : écriture de l'échantillon, somme de convolution, avancée de l'index. Source « impulsions », noyau `noyauRetard(h, 100)` : l'impulsion de la voie 1 doit être décalée par rapport à celle de la voie 2. Que se passerait-il si l'on oubliait de faire avancer `w_` ?

```
for (int n = 0; n < CONV_L; n++) {
  // TODO (Q3) 1) écrire l'échantillon courant x[n] (normalisé : / 32768.0f)
  //              dans le buffer circulaire, à l'index w_

  // TODO (Q3) 2) y = somme pour k = 0 .. M_-1 de h_[k] · x[n - k],
```

```
6     //              où x[n - k] est rangé dans circ_[(w_ - k) & CIRC_MASK]
7     float y = block->data[n] / 32768.0f;           // provisoire : pas de filtrage
8
9     // TODO (Q3) 3) faire avancer w_ d'une case, avec retour à 0 après CIRC_N - 1
10
11    block->data[n] = versInt16(y);
12 }
```

3 Lire une réponse impulsionnelle

Avec la source « impulsions », la voie 1 de l'oscilloscope montre directement $h[n]$, un échantillon toutes les 22,7 μs. Le filtre de reconstruction du convertisseur arrondit un peu chaque impulsion isolée (quelques dizaines de microsecondes) : on lit les durées et les amplitudes, pas la forme exacte d'un échantillon.

Activité 2

Q4. Source « impulsions ». Pour chaque noyau (une ligne à choisir dans `setup()`), relever la réponse à l'oscilloscope et l'expliquer :

(a) **retard** $h = \delta[n - D]$, $D = 100$: mesurer le décalage entre les voies et le comparer à D/F_s ;

(b) **écho** $h = \delta[n]+g\,\delta[n-D]$, $D = 2000$, $g = 0{,}5$: écart et amplitudes des deux impulsions ? Combien de répétitions entend-on sur la musique ?

(c) **moyenne** sur 64 points : durée et hauteur de la réponse ?

(d) **passe-bas** à 1 kHz, 255 coefficients : quelle forme reconnaît-on ? Durée du lobe central ?

(e) **salle**, 3968 coefficients : à quoi ressemble une réponse de pièce ? L'écouter sur la musique.

Q5. Noyau « moyenne » sur 64 points, source « sinus ». Prévoir les fréquences que ce filtre annule. Vérifier à l'oscilloscope en réglant `sinus.frequency()` sur la première d'entre elles, puis sur 1000 Hz.

4 Coût de calcul

Activité 3

Q6. Source « musique », noyau `noyauSalle(h, M)`. Relever la charge affichée dans le moniteur série pour chaque longueur M du tableau et tracer la courbe. Est-elle linéaire ? En déduire le nombre de cycles par MAC et la longueur maximale utilisable. Combien de coefficients faudrait-il pour un écho de 1 s ? Est-ce envisageable ?

M	16	64	256	1024	2048	3968
CPU (%)						

5 Livrables

1. Convolution directe fonctionnelle, historique rangé dans un buffer circulaire ;
2. Relevés et explication des cinq réponses impulsionnelles à l'oscilloscope ;
3. Courbe de charge processeur en fonction de M et longueur maximale.

® **Remarque** : Pour aller plus loin

Comparer avec l'objet `AudioFilterFIR` de la bibliothèque (calcul en virgule fixe, CMSIS) ; mesurer le gain de vitesse.